ML Assignment 4

UNIVERSITY OF HARTFORD

Assignment 4

Support Vector Machines

Evaluation of kernel technique performance in support vector machine classification models.

Student: John Bennett

Course: CS569 Machine Learning

Fall 2025

October 30, 2025

Contents

1 Objective **1**

2 Data Set Overview and Cleaning **1**

3 Approach **1**

4 Results and Discussion **2**

5 Conclusion **4**

1 Objective

This assignment focused on comparative analysis between kernel techniques deployed in support vector machine models. In this context C and $Gamma$ hyperparameters were controlled for effect on models built with different kernel techniques. Procedures included:

- Establishing synthetic source data for training and testing of support vector machine models
- Establishing and storing training and testing results for combinations of different hyperparameter value sets, focusing on kernel techniques
- Parsing training and testing results into interpretable forms for identifying which kernel led to the best models

2 Data Set Overview and Cleaning

Analysis in this assignment used a synthetic dataset generated through use of the "make_classification" method of Sci-kit Learn's 'dataset' module in Python. The dataset modeled highly variant numeric features consistent with that produced by scientific instrumentation. It consisted of 1250 records and 11 features, including a binary target feature for supervised training of the classification models. The 10 training features were all informative for model training and consisted of no redundant features. Complexity was added to the features by establishing 3 feature-space clusters per class with noise established through flipping of classes of records in 3% of the data and class stratification in a 35% to 65% class distribution across records. This dataset was designed to not require cleaning and the abstract features allow for focus on the efficacy of model techniques themselves.

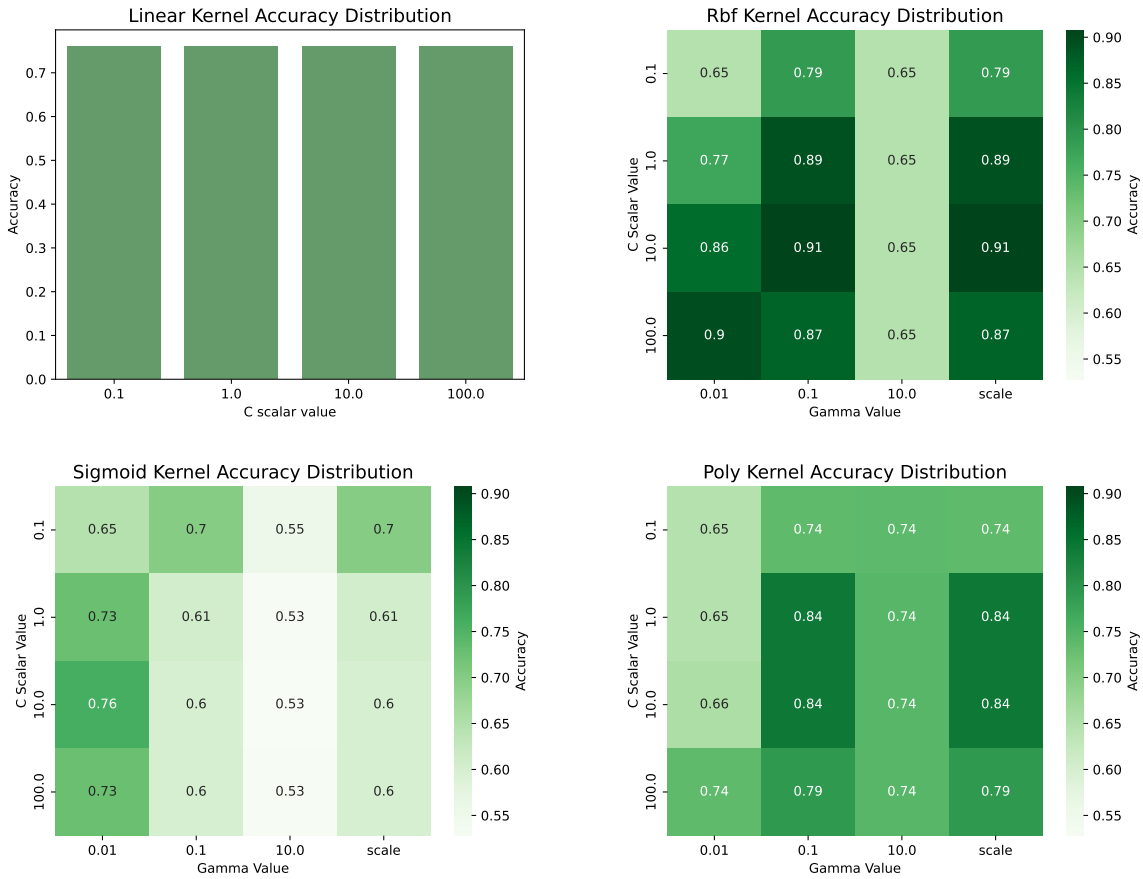
3 Approach

Analysis of different Support Vector Machine Classifiers (SVCs) model training options was focused on the the type of kernel techniques while also accounting for the effects of other hyperparameters utilized in training SVCs. Kernel techniques offer interventions in the model training process by transforming training feature vectors into greater value ranges or higher dimensional spaces to allow for the decision boundaries in the model to be more easily established, and ideally have less model complexity. They are essential for handling data sets where the data objects in their feature space are not linearly separable across their classifications. The boundaries themselves remain linear while the feature space is transformed. The kernels used included linear, polynomial, radial basis function (Gaussian), and sigmoid.

Models were trained with each of these kernels being deployed with combinations of values for the C and Gamma hyperparameters. The pool of values for these parameters were $C = [0.1, 1, 10, 100]$ and $Gamma = [scale, 0.01, 0.1, 10]$, and all combinations were exhausted providing a comprehensive perspective for the performance of each kernel overall with the different hyperparameters explored.

4 Results and Discussion

Figure 1: Model Accuracy Across Kernel Types



The plots in figure 1 show how each kernel performed overall, providing a broad perspective on the accuracy associated with each and how that accuracy varied based on the values used for the C and Gamma hyperparameters. The goal of the SVC algorithm is to minimize the vector $\boldsymbol{w}$ so that the decision boundary margin is maximized while also minimizing the overall instances of data objects falling within this margin in the vector space. Briefly, $\boldsymbol{w}$ is the weighted vector sum of each support vector that plays a role in the decision boundary with an implication being that the smaller its value, the wider the margin around the decision boundary. Both C and Gamma effect the magnitude of costs associated with data points that violate the margin of the decision boundary; they influence the extent to which the margin will be maximized and to what extent data violating that margin will be tolerated so the tuning of value of these hyperparameters can be impactful on the accuracy of a model.

The top left bar plot identifies the accuracy distribution for the linear kernel models. The linear kernel does not have the Gamma hyperparameter associated with it, only the C hyperparameter. As a result of the single hyperparameter being controlled, and the fact that the accuracy of the linear kernel models had a uniform distribution across all tested values of C, it was most effective to use a bar plot to show how little impact C had on the performance of linear kernel models. For all linear kernel models, the accuracy was a constant 76%.

The remaining plots were heat maps that showed the accuracy distribution of for each kernels relative to the values of C and Gamma in a specific model. The radial basis function (RBF) kernel models (top right) had the highest accuracy scores in their distribution, followed by the polynomial kernel models, the linear kernel models, and finally the sigmoid kernel models. The highest accuracy was the RBF kernel model with a $C = 10$ (3rd row from the top) and a $Gamma = 0.1$ (second column from the left). The next four highest scores where also RBF kernel models. The polynomial kernel's accuracy distribution was notable for having the most stable accuracy scores, given that its distribution was not as adversely effected by $Gamma = 10$ as the Sigmoid and RBF kernel models were. In addition to having the lowest overall performance, the performance for models with $C = 10$ performed right around that of random selection. One final trend to note is that the accuracy distributions of $Gamma = .10$ and $Gamma = scale$ are identical in all of the kernel types.

Table 1: Top Accuracy

Kernel Type	C	Gamma	Test Accuracy	# Support Vectors	Training Time
rbf	10.0	0.1	0.908	[162, 169]	0.011187
rbf	10.0	scale	0.908	[162, 169]	0.010393
rbf	100.0	0.01	0.896	[181, 190]	0.019483
rbf	1.0	scale	0.892	[223, 240]	0.009249
rbf	1.0	0.1	0.892	[223, 240]	0.009488
rbf	100.0	scale	0.872	[124, 146]	0.016195
rbf	100.0	0.1	0.872	[124, 146]	0.012709
rbf	10.0	0.01	0.860	[243, 248]	0.010582
poly	1.0	0.1	0.840	[251, 270]	0.010617
poly	10.0	0.1	0.840	[182, 200]	0.014992
poly	10.0	scale	0.840	[182, 200]	0.015713
poly	1.0	scale	0.840	[251, 270]	0.010578

Table 2: Top Training Time

Kernel Type	C	Gamma	Test Accuracy	# Support Vectors	Training Time
rbf	1.0	scale	0.892	[223, 240]	0.009249
rbf	1.0	0.1	0.892	[223, 240]	0.009488
poly	100.0	0.01	0.736	[330, 342]	0.009980
rbf	10.0	scale	0.908	[162, 169]	0.010393
poly	1.0	0.01	0.648	[352, 357]	0.010422
poly	0.1	0.1	0.736	[330, 342]	0.010531
poly	1.0	scale	0.840	[251, 270]	0.010578
rbf	10.0	0.01	0.860	[243, 248]	0.010582
poly	1.0	0.1	0.840	[251, 270]	0.010617
poly	10.0	0.01	0.656	[352, 365]	0.010652
linear	0.1	-	0.760	[296, 297]	0.010693
rbf	1.0	0.01	0.772	[312, 318]	0.010801
poly	0.1	0.01	0.648	[352, 352]	0.010835
rbf	10.0	0.1	0.908	[162, 169]	0.011187
rbf	0.1	0.1	0.788	[341, 354]	0.011607

Table 3: Top Complexity

Kernel Type	C	Gamma	Test Accuracy	# Support Vectors	Training Time
poly	0.1	10.0	0.740	[92, 126]	4.990075
poly	1.0	10.0	0.744	[94, 124]	7.166061
poly	100.0	10.0	0.744	[94, 124]	7.238293
poly	10.0	10.0	0.744	[94, 124]	6.955204
rbf	100.0	scale	0.872	[124, 146]	0.016195
rbf	100.0	0.1	0.872	[124, 146]	0.012709
poly	100.0	0.1	0.792	[135, 145]	0.050299
poly	100.0	scale	0.792	[135, 145]	0.049933
rbf	10.0	0.1	0.908	[162, 169]	0.011187
rbf	10.0	scale	0.908	[162, 169]	0.010393
rbf	100.0	0.01	0.896	[181, 190]	0.019483
poly	10.0	0.1	0.840	[182, 200]	0.014992
poly	10.0	scale	0.840	[182, 200]	0.015713
sigmoid	100.0	scale	0.600	[213, 216]	0.013552
sigmoid	100.0	0.1	0.600	[213, 216]	0.013270

Table 1 again shows the extent to which the RBF kernel outperforms the the other kernels. The table consists of model with accuracy above 80% with RBF kernel models encompassing the majority of the table. Table 2 shows the best performing models with respect to time. The fact that there are 7 of the top 12 most accurate models shown in Table 1 shows that time complexity is not necessarily a trade off with predictive performance. Table 3, which shows the models with the lowest number of support vectors (and therefor the least complexity), reiterates this point since again, the best performing model is also found in this set of models. It is notable that the models with the least complexity took some of the longest times to train overall, indicating that model complexity and training time are not directly correlated.

5 Conclusion

Conclusively, the radial basis function (RBF) kernel models performed the best overall with this particular training data, especially when omitting $Gamma = 10$. Its best performing model ($kernel = RBF, C = 10, Gamma = 0.1 = "scale"$) sacrificed very little with respect to training time efficiency or model complexity while achieving its accuracy. This shows that simply increasing the values of hyperparameters does not necessarily serve the best outcome. This can also be seem in the argument values for C where $C = 10$ outperformed $C = 100$. The conclusions of this experimentation demonstrate that there is often not a clear universal technique that out performs all others. In this case, RBF was the most effective kernel method but, there is still a need with model training to be thorough about getting to the model that best reflects the patterns in the particular group of entities that training data is a proxy for. The relevant assumption of machine learning seems to be that future entities from the group represented in the training data will match the patterns that the model was built on; it is not the assumption that conclusions of prior modeling procedures can necessarily be projected onto the assumptions of future modeling procedures.